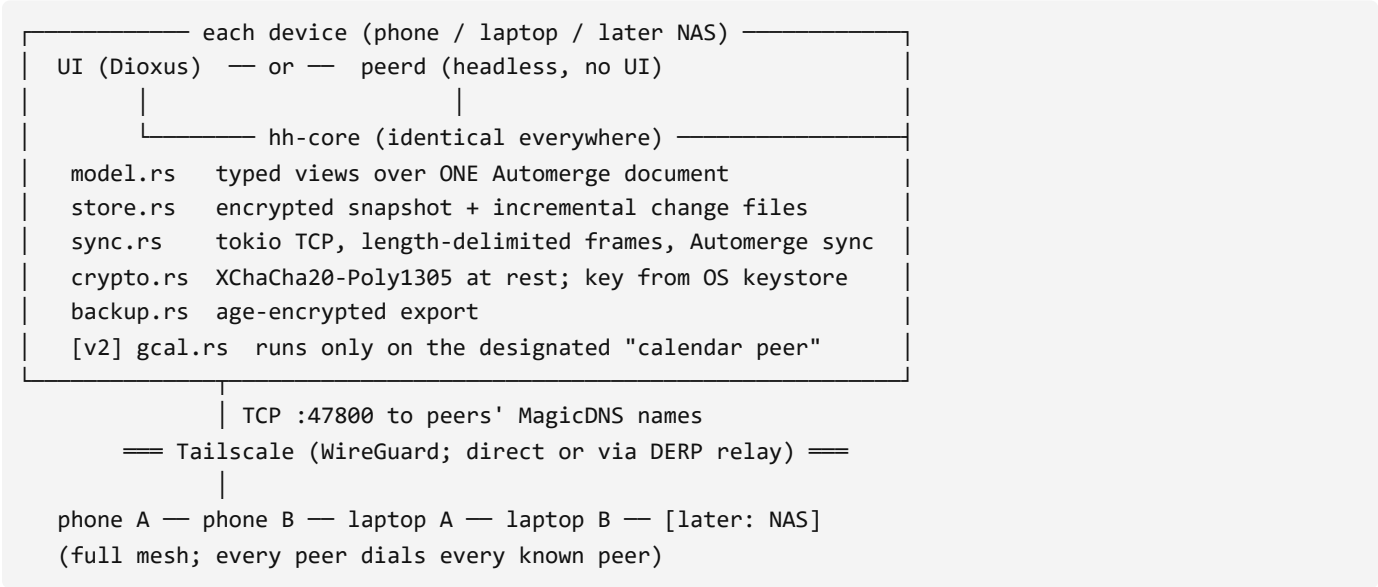
Household Organiser — Plan

Local-first, peer-to-peer household organiser for two people. Rust. No server; peers sync over a Tailscale tailnet.

Decisions so far

Topic	Decision
UI	All-Rust → **Dioxus**
Google Calendar	Push app items to a dedicated calendar + read-only view of other events. **v2**
Budget	Manual entry only. AUD only. Multiple shared accounts (spending / saving / credit card); track *which account*, not *who paid*. Grocery shops record one total per trip, not per-item prices
Tasks	Assignable to person A, person B, joint, or unassigned. Everyone sees every task; only the assignee's devices notify
Deletes	Hard deletes for v1
Android	**Real background sync** via foreground service
Backups	Encrypted snapshot to a user-chosen folder

1. Architecture



2. Cargo workspace

```
Cargo.toml            # [workspace], exact "=" pins for dioxus/automerge/autosurgeon
rust-toolchain.toml   # pinned stable channel
crates/
  hh-core/            # lib: model, store, sync, crypto, backup. No UI or platform deps.
  hh-app/             # bin: Dioxus UI for Windows + Android (Linux desktop later)
    android/          # Kotlin glue (§7), wired in via Dioxus.toml
  hh-peerd/           # bin: headless peer, TOML config. ~100 lines over hh-core
```

Three crates, deliberately. The only platform difference is the keystore. The binary passes a `KeyProvider` into `hh-core` at startup; no other traits.

3. UI framework

	Dioxus 0.7	**Slint**	**Tauri 2 (reference)**
UI language	Rust (RSX) + CSS, rendered in system WebView	Rust + `.slint` DSL, native render	TS/HTML
Android maturity	Usable, young. 0.7 added AndroidManifest/MainActivity.kt customisation	Supported, no plugin ecosystem	Most mature, official plugins
Android notifications / background	**None built in.** Kotlin/Java FFI is on the 0.8 roadmap, no date	Same: hand-written JNI	Notification plugin exists; foreground service still custom Kotlin
Non-Rust code	Kotlin glue + CSS	Kotlin glue	TS + Kotlin

Choice: Dioxus. It's all-Rust, and hot-reload on a real device should be the main productivity win. The Android foreground service is custom Kotlin in *every* option, Tauri included, so all-Rust adds little Kotlin.

Biggest risk: it's unconfirmed that Dioxus 0.7's generated Gradle project accepts extra Kotlin files and a `<service>` entry cleanly. Phase 0 tests this first. If it fails, eject the generated Gradle project and maintain it by hand; Slint is the second choice. Dioxus ships breaking minor releases (example), so use exact pins, and keep `dioxus-cli` on the same version as the `dioxus` crate.

4. CRDT and data model

Choice: **`automerge`** (0.11 on docs.rs) + `autosurgeon` for derive-based struct↔doc mapping. Chosen over `yrs` because:

- **Built-in sync protocol** (`automerge::sync`: `generate_sync_message` / `receive_sync_message`), so there's no protocol to design. `yrs` gives state-vector diffs, but the session logic is DIY.
- JSON-like map/list/**counter** model. Counters handle concurrent "used one" decrements correctly.
- Full history allows `fork_at(heads)`, which helps recover from bad edits.
- `yrs` is stronger at collaborative rich text, which this app doesn't need.

Granularity: one doc for the whole household. The data is small, and "finish shop → checked items into inventory + one budget transaction for the total" is **one atomic change**. There's one sync state per peer. *Ceiling:* split the budget into one doc per year if load time suffers (history in the tens of MB).

Layout. Every collection is a **map keyed by UUID, never a list**, so there are no list-index conflicts.

```
root
  schema_version: u32
  people:   {a: {name}, b: {name}}                      # exactly two
  devices:  {device_id → {name, magicdns, owner: a|b|none}}   # also the peer list (§5)
  tasks:    {id → {title, notes, due: Option<date|datetime>, done_at,
                   assignee: a|b|joint|unassigned, created_by: a|b}}
  lists:    {id → {name, kind: "shopping"|"general", items: {id → {text, qty, checked}}}}
  inventory:{id → {name, qty: Counter, unit, location: fridge|pantry|freezer|other,
                   expiry: Option<date>}}
  budget:
    accounts:   {id → {name, kind: spending|saving|credit, opening_balance_cents: i64}}
    categories: {id → {name, monthly_limit_cents: i64}}
    txns:       {id → {date, kind: expense|income|transfer, amount_cents: i64 (>0),
                       account, to_account: Option (transfer only),
                       category: Option (expense only), note, source_list: Option<id>}}
```

Budget semantics (AUD only, integer cents, never floats):

- **Expense** on an account counts against its category for that month. A **credit-card purchase is an expense on the credit account** at purchase time.
- **Transfer** (paying off the card, moving money to savings) never counts as spending. This avoids double-counting card purchases when the card is paid.
- **Account balance** = opening + income − expenses ± transfers. A credit account's balance is negative when money is owed.
- Budget view per month: spent vs limit per category, plus account balances.
- **Grocery spending is recorded per shop, not per item.** Ticking items off records no prices. "Finish shop" on a shopping list asks for **one total + account** (account defaults to the last one used on that device, category defaults to Groceries). It then creates a single expense with `source_list` set, moves the checked items into inventory, and clears them from the list, all in one change. Skipping the total is allowed (e.g. someone else paid, or it was entered manually).

Merge behaviour:

- **Concurrent edits** merge per field. The same field edited at once is last-writer-wins for that field only. The losing value is kept (`get_all`) if it's ever worth showing.
- **Deletes** are hard deletes of the map key. If one person deletes an item while the other edits one of its fields, the edit is lost. Accepted for v1; backups and history cover mistakes.
- **Schema evolution:** additive only. New fields are `Option` or have a default, and keys are never renamed. If a peer sees a `schema_version` newer than it understands, it goes **read-only** and shows "update the app".
- **Genesis gotcha:** if two devices each create `root.tasks`, Automerge ends up with competing maps and one disappears. Only the first device initialises the doc. New devices start empty and **can't write until their first sync completes**. Every install gets a fresh actor ID, and a restored backup never reuses one.

Inventory and budget fit in v1 in trimmed form. Min-stock auto-add is v2. When it comes, the restock item's ID is derived from the inventory ID (`restock:<inv_id>`), so two devices adding it at once merge into one entry instead of creating duplicates.

5. Sync protocol and discovery

- **Transport: plain `tokio` TCP.** WireGuard already encrypts and authenticates. `quinn`/QUIC would add a second TLS layer and certificate handling for nothing; its multiplexing doesn't matter for one doc. TCP over DERP is just slower, which is fine.
- **Framing:** `tokio_util::codec::LengthDelimitedCodec` with `max_frame_length` = 16 MiB. Frames are `postcard`-encoded `enum Msg { Hello{proto_ver, device_id}, Sync(Vec<u8>) }`.
- **Session:** after Hello, exchange Automerge sync messages until both sides have nothing to send. The connection **stays open**: every local change pushes a sync message to each connected peer, so sync is automatic. Dropped connections reconnect with exponential backoff (max 60 s). If two peers dial each other at the same time, the connection opened by the lower device ID is kept.
- **Discovery:** a **static peer list of MagicDNS names**, kept in the shared doc (`root.devices`), so adding a peer once teaches every device. Tailscale LocalAPI is skipped: it means platform-specific code (and on Android it isn't reachable from other apps) for a 5-device list.

6. Storage and encryption at rest

Choice: files encrypted with XChaCha20-Poly1305 (`chacha20poly1305`). No SQLite. Automerge *is* the database.

- On disk: `doc.snapshot.enc` (from `save()`) + `changes/NNNN.enc` (from `save_incremental()`). Change files are folded into a new snapshot once there are more than 100.
- Writes go to a temp file, then rename. The filename is the AAD.
- **SQLCipher** (`rusqlite bundled-sqlcipher`) is rejected: it means cross-compiling OpenSSL or another crypto backend for Android from Windows, for a table we don't need.

Key: 32 random bytes per device, held in the OS keystore:

- **Windows:** Credential Manager via the `keyring-core` ecosystem's Windows store. The crate family was restructured in v4; check the exact store-crate names on docs.rs when implementing.
- **Android:** Kotlin creates an AES key in **AndroidKeyStore** and uses it to wrap the data key. The wrapped key lives in app-private files.
- **Linux desktop (later):** Secret Service via the same ecosystem.
- **Headless NAS:** a key file with mode `0600`. The real protection there is full-disk encryption (LUKS).

Android file-based encryption and BitLocker already cover a lost, locked device. This layer is defence in depth. Set `android:allowBackup="false"`.

7. Android background strategy (Kotlin cost)

- **Foreground service type `specialUse`, not `dataSync`.** On Android 15+, `dataSync` is capped at **6 h per 24 h** and can't start from `BOOT_COMPLETED`. `specialUse` has no timeout but needs **Play Console review**, which doesn't apply when sideloading onto two phones. *This depends on never publishing to Play.*
- The service starts on app launch and after boot. It shows a **permanent low-priority notification**; that's the accepted trade-off.
- The Rust core is a process-wide singleton. Whichever starts first (Activity or Service) starts it, and the UI attaches.
- **Due-task notifications:** the core is always running, so a once-a-minute Rust check calls Kotlin to post them. No AlarmManager, no exact-alarm permission. A device notifies for a task when

`devices[self].owner` matches the assignee. **Joint** notifies both people. **Unassigned** notifies both by default (see open decisions). Devices with owner `none` (the NAS) never notify.

- **Kotlin glue, roughly 250–350 lines:** Service (~100), JNI entry points (~50), keystore wrap/unwrap (~70), notification posting (~50), permission requests for notifications and the battery-optimisation exemption (~50).
- **Battery:** idle TCP over a WireGuard tunnel Tailscale already keeps open. Expect a few % per day; measure it in Phase 0. The real cost is the always-visible notification.
- **Windows laptops:** closing the window hides the app to the tray, and it starts at login. Notifications use `notify-rust` (WinRT toasts).

8. Google Calendar (v2)

- **One calendar peer**, set in `root.gcal.owner_device`: a laptop in v2, the NAS later. Only that peer holds a refresh token. It lives in that peer's keystore and is **never synced**.
- **Idempotent event IDs:** Google requires base32hex (`a-v0-9`), 5–1024 chars. Hex is a subset of base32hex, so the ID is `hh` + UUID hex, with `+ yyyymmdd` added for recurring occurrences. Writes are **upserts**: try `update`, and on 404 do `insert`. Google doesn't guarantee it detects duplicate IDs, which is the second reason only one peer runs the calendar sync.
- **Read-only view:** the calendar peer pulls a ±60-day window of other calendars into `root.gcal_cache`, keyed by event ID and written diff-only so history grows slowly. Past events are pruned.
- **OAuth:** `oauth2` with loopback redirect + PKCE, plus `reqwest + serde` for ~4 endpoints. The generated `google-calendar3` crate is skipped as unnecessary weight. Google's **device flow doesn't allow calendar scopes** (allowed list), so the headless NAS authorises with `ssh -L` forwarding the loopback port.
- **7-day expiry:** set the consent screen to **"In production"**, not "Testing". Unverified apps keep working for a small number of users behind a warning screen. **Re-consent once** after switching to get a non-expiring token.

9. Security

- **Peer authentication** rests on Tailscale plus two cheap checks:

  1. A **Tailscale ACL** allowing only `tag:household → tag:household:47800`. This matters if the tailnet also holds homelab nodes.
  2. A **source-IP allowlist.** Tailscale binds a node's 100.x source IP to its key, so checking the peer's IP against the resolved peer list is real authentication.

  There's no app-level crypto handshake in v1. If nodes are ever shared into the tailnet, add a pre-shared-key HMAC challenge.

- **Untrusted input:**

  - 16 MiB frame cap, 30 s read timeout, at most 1 connection per peer. Any `receive_sync_message` error drops the connection.
  - Incoming changes are merged into **`doc.fork()` first**. The fork is hydrated into the typed model and checked against limits: string lengths, item counts, no unknown `schema_version` jump. Only if that passes is it merged into the real doc and saved.
  - Limit: a compromised peer can still make *valid* destructive edits. History and backups are the recovery path.

- **Lost device:** expire it in the Tailscale admin console to cut sync immediately, then remove it from `root.devices`.

10. Durability, bootstrap and backups

- **New device:** install the app, enter one peer's name, and it syncs the full doc. Writes are blocked until that first sync completes (genesis gotcha, §4).
- **Backups:** one laptop (later the NAS) writes `household-YYYYMMDD.am.age` daily to a chosen folder and keeps the last 30. OneDrive is fine; it only ever sees ciphertext. They're encrypted with an `age` passphrase that is **independent of device keys**, so restoring works on brand-new hardware.
- **Restore:** merging a backup is the normal restore, and it's idempotent. To roll back, write fields back from `fork_at(old_heads)` as a new change.

11. Toolchain and builds

- **Windows: MSVC** (the rustup default). WebView2 and most `-sys` crates assume it; GNU only adds friction.
- **Android:**
  - Install the NDK and JDK 17 through Android Studio's SDK Manager.
  - Set `ANDROID_HOME`, `ANDROID_NDK_HOME` and `JAVA_HOME`.
  - `rustup target add aarch64-linux-android` (add `x86_64-linux-android` for the emulator).
  - `dx bundle --platform android` drives Gradle (Dioxus mobile guide).
- **Linux headless:** pure Rust (no SQLCipher or OpenSSL), so `cargo zigbuild --target x86_64-unknown-linux-musl` (cargo-zigbuild) or `cargo build` in WSL2.
- **Pinning:** commit `Cargo.lock`, use `=` pins for `dioxus*`, `automerge` and `autosurgeon`, and pin the `rust-toolchain.toml` channel.
- **Changelogs to check before bumping:** Dioxus releases, automerge releases.
- All peers run the same app version. `proto_ver` in Hello rejects a mismatch with a clear error.

12. Future always-on peer

`hh-peerd` = `hh-core` + a TOML config (peer list, data dir, key file, backup dir, `gcal_owner`), run as a systemd unit. Add its MagicDNS name to `root.devices` with owner `none`. It then becomes the always-on relay between the phones and the natural owner of backups and calendar sync. No redesign.

13. Roadmap

Phase 0: spikes, go/no-go, about a weekend each

1. Dioxus Android hello-world, plus a custom Kotlin `specialUse` service that calls into Rust over JNI and survives the app being swiped away. **If this fails, revisit the UI choice before anything else.**
2. Two `hh-core` peers (laptop + phone) syncing a toy Automerge doc across the tailnet, including over DERP. Measure battery overnight.

v1: daily-usable

- Tasks with and without due dates, with assignee (A / B / joint / unassigned) and assignee-only notifications.
- Shopping and general lists.

- Inventory (qty, location, optional expiry). Checking off a shopping item moves it into inventory.
- Budget: accounts (spending/saving/credit), categories with monthly limits, manual expense/income/transfer entry, "Finish shop" records one total + account per grocery trip.
- Automatic sync, full offline use.
- Android foreground service, local notifications.
- Encrypted storage, daily encrypted backup.
- Windows tray + autostart.

v2

- Google Calendar (push + read-only view).
- Recurring routines: every N days/weeks/months, **fixed or from completion**. No full RRULE.
- Min-stock auto-add.
- Trash/undo view.

Later

- `hh-peerd` on the NAS.
- Linux desktop.
- Theming.
- CSV import.
- Per-year budget docs.
- PSK handshake.
- Extra list types (watchlist, games) are **already covered** as named general lists; at most they need an icon field.

14. Risks

1. **Dioxus on Android**: adding Kotlin and a service to the generated project. Phase 0 tests it first.
2. Dioxus and Automerge ship breaking minor releases. Bumps are deliberate events, with pins.
3. `specialUse` ties you to sideloading. Publishing to Play later would need a review justification or a redesign.
4. The single doc grows with history. Monitor its size; the per-year budget split is the escape hatch.
5. Valid-but-destructive edits from a buggy peer spread everywhere. Backups are the backstop.
6. Laptops only sync while the app runs. Until the NAS exists, phone↔phone sync needs both phones online at the same time, which the foreground service makes likely.

15. Open decisions

1. **Unassigned tasks:** notify both people (current default) or nobody?
2. **Distribution:** confirm sideload-only. This is what makes `specialUse` workable.
3. **Backup passphrase:** where will you keep it (e.g. password manager)?
4. **Phase 0 fallback** if Dioxus Android fails: eject and maintain the Gradle project by hand, or switch to Slint?
5. **Budget periods:** calendar months assumed. Say so if you budget by pay cycle (e.g. fortnightly).